**Blockchain Technologies 2 Final Project Report**
**Option E - Decentralized Insurance Pool**
**Architecture Document**

Team roles:
Zhambyl Dulat - smart contracts, integration, deployment scripts, final repository;
Murat - frontend integration and subgraph structure;
Daulet - tests, coverage, audit notes, gas report.

Repository: `dooolat/decentralized-insurance-pool`

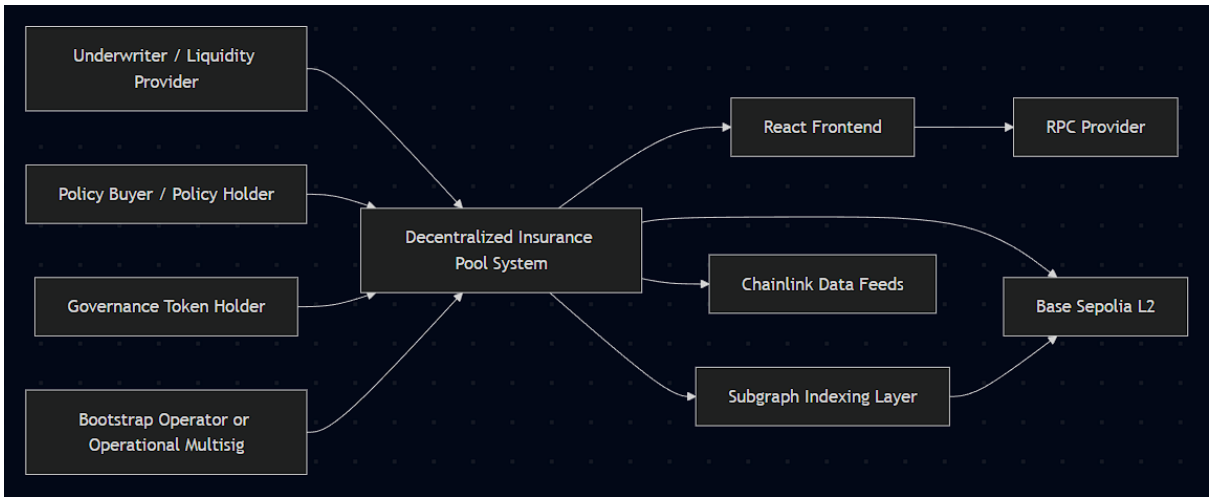
Reviewed branch state: `develop`

Reviewed commit: `9c22d97938c80048911199ecc7737416b3e3664b`

This document describes the architecture of the Option E Decentralized Insurance Pool project as it exists in the reviewed `develop` branch. It is written to accompany the smart-contract repository rather than to replace the code. When this document mentions deployed infrastructure, it refers only to the recorded Base Sepolia deployment stored in `deployments/base-sepolia.json`. It does not claim mainnet deployment, a live hosted subgraph endpoint, or a fully production-hardened frontend wallet flow.

## 1. System Context Diagram (C4 Level 1)

At C4 Level 1, the protocol is treated as one system boundary with external actors and dependencies around it.



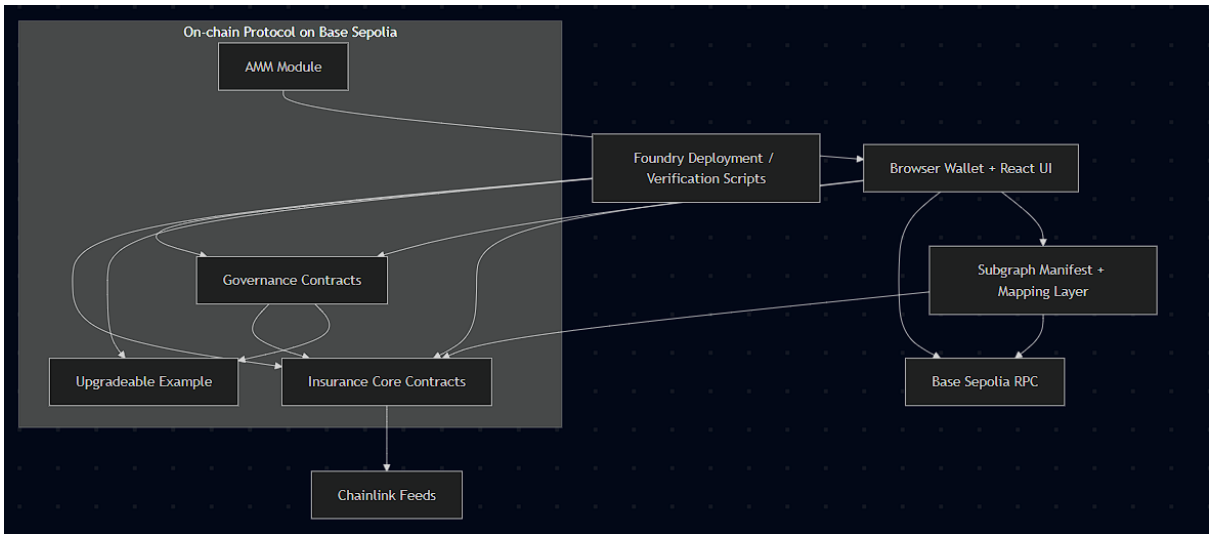
**System Context Notes**

- The system provides three user-facing roles:
- underwriters deposit collateral into the vault;
- buyers purchase coverage and receive policy NFTs;
- governance token holders propose and vote on privileged changes.
- The protocol runs on an L2 chain, currently Base Sepolia for the validated deployment record.
- Chainlink is an external trust dependency for claim trigger data.
- The frontend and subgraph are convenience layers. They improve operability, but they are not consensus-critical.
- Bootstrap operators are operational actors outside the ideal steady-state governance model. Their authority should shrink as ownership and roles move to the timelock/governor path.

## 2. Container and Component Architecture

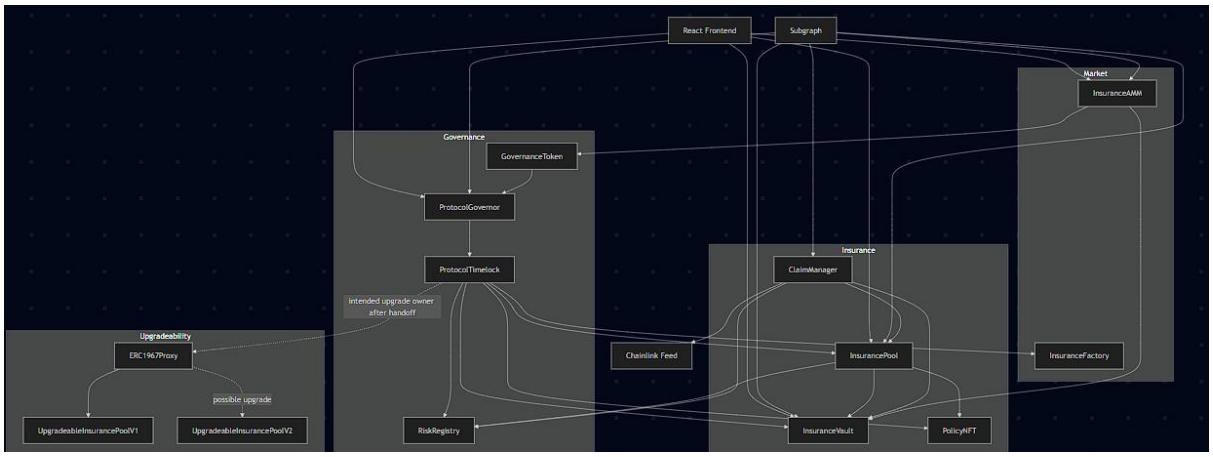
### 2.1 Container Diagram

The protocol can be decomposed into user-facing, indexing, execution, and governance containers.




### 2.2 Component Diagram

The component view below focuses on contract relationships, proxy layout, access-control boundaries, and external dependencies.




## 2.3 Component Responsibilities

| Component | Responsibility | Key Dependency |
|---|---|---|
| `GovernanceToken` | vote weight, delegation, quorum basis | `ProtocolGovernor` |
| `ProtocolGovernor` | proposals, voting, queueing, execution orchestration | `ProtocolTimelock` |
| `ProtocolTimelock` | delayed execution gate for privileged calls | owner/role-controlled modules |
| `RiskRegistry` | source of risk parameters and oracle feed selection | Chainlink feed addresses |
| `InsuranceVault` | ERC-4626 collateral accounting and reserve-aware liquidity checks | `InsurancePool`, collateral token |
| `InsurancePool` | policy lifecycle, premium transfer, coverage reservation | `PolicyNFT`, `RiskRegistry`, `InsuranceVault` |
| `ClaimManager` | oracle-triggered claim checks and payout execution | `InsurancePool`, `ChainlinkOracleAdapter`, `InsuranceVault` |
| `PolicyNFT` | policy receipt NFT minting and metadata | `InsurancePool` |
| `InsuranceAMM` | constant-product swap and LP accounting | governance token and collateral token |
| `InsuranceFactory` | owner/timelock-gated CREATE/CREATE2 deployment helper | arbitrary bytecode |

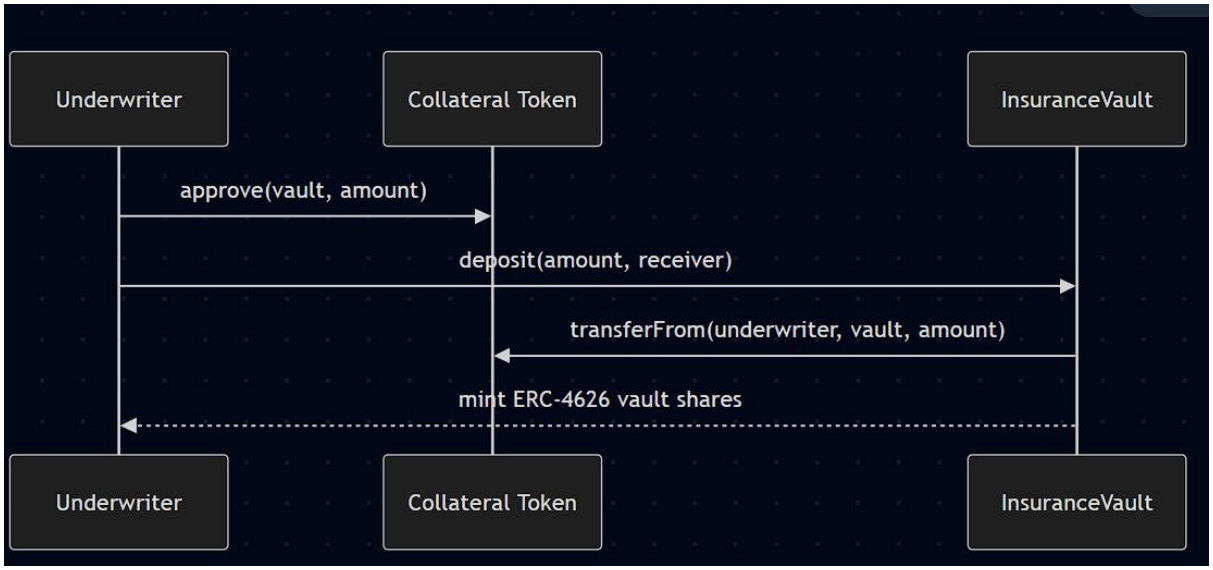
## 2.4 Access-Control Topology

| Control Surface | Current Contract Model | Operational Meaning |
|---|---|---|
| `PolicyNFT.setInsurancePool` | `onlyOwner` | owner can repoint the only minter |
| `PolicyNFT.setBaseTokenUri` | `onlyOwner` | owner can mutate NFT metadata base URI |
| `InsuranceVault.setInsurancePool` | `onlyOwner` | owner wires reserve source |
| `InsuranceVault.setClaimManager` | `onlyOwner` | owner wires payout authority |
| `InsuranceVault.pause/unpause` | `onlyOwner` | owner can freeze deposits and exits |
| `InsurancePool.setClaimManager` | `onlyOwner` | owner wires claim executor |
| `InsurancePool.pause/unpause` | `onlyOwner` | owner can freeze policy actions |
| `RiskRegistry.add/update/deactivateRiskType` | `onlyRole(RISK_MANAGER_ROLE)` | role holder can change insured risk definitions |
| `InsuranceFactory.deployCreate/deployCreate2` | `onlyOwner` | owner can deploy arbitrary bytecode |
| `UpgradeableInsurancePoolV1._authorizeUpgrade` | `onlyOwner` | proxy owner controls upgrade path |
| `ProtocolGovernor` | token-weighted voting | governance defines delayed privileged actions |
| `ProtocolTimelock` | role-based delayed executor | timelock is the intended final control boundary |

## 3. Critical User Flows

The assignment requires at least three critical flows. This project documents five, because the protocol meaningfully spans underwriting, insurance purchase, claim execution, market interaction, and governance.

### 3.1 Underwriter Deposit Flow

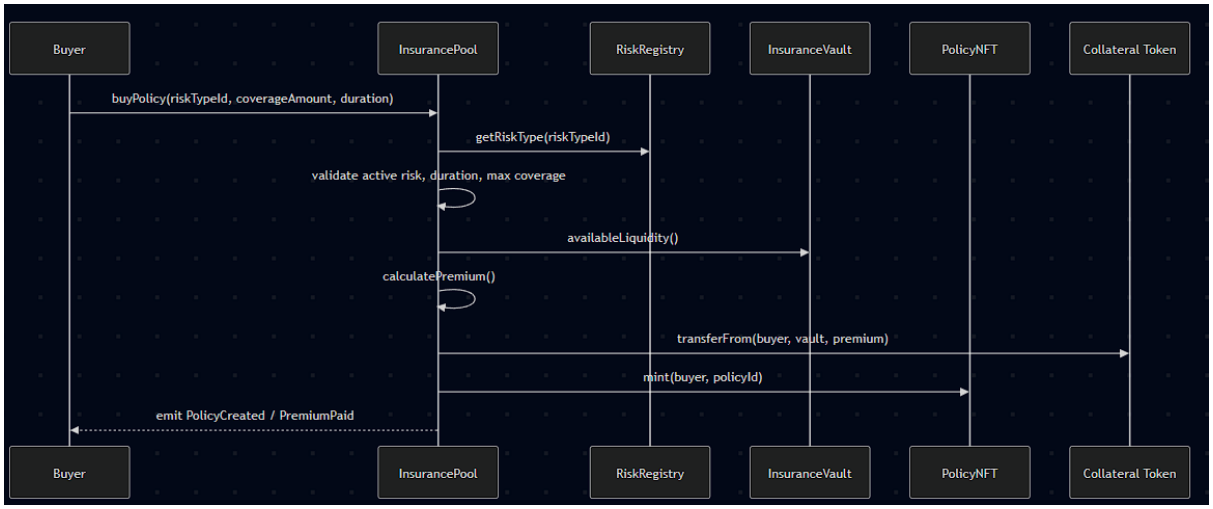



Security-relevant invariants:

- deposits are blocked when the vault is paused;
- minted shares follow ERC-4626 math rather than custom bookkeeping;
- the vault keeps the underlying collateral and later uses `activeCoverage`

reported by the pool to decide how much is free to withdraw.

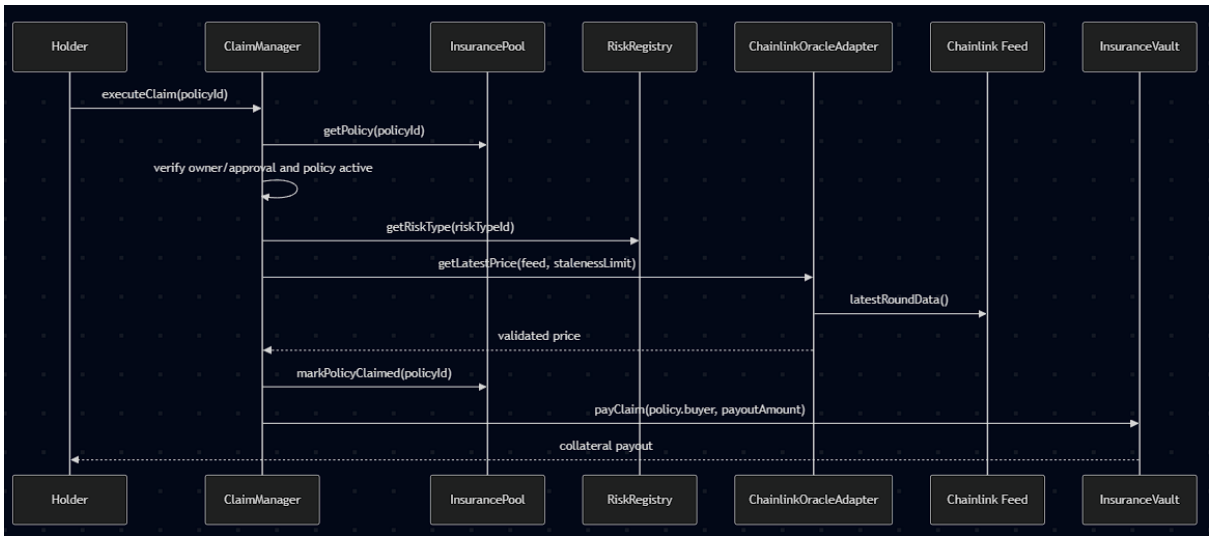
### 3.2 Policy Purchase Flow




Security-relevant invariants:

- inactive risks cannot be purchased;
- coverage must fit both the risk-specific cap and free protocol liquidity;
- the premium is moved straight into the vault, not parked in the pool;
- the pool increments `activeCoverage` before later claims/expiries release it.

### 3.3 Oracle-Triggered Claim Flow

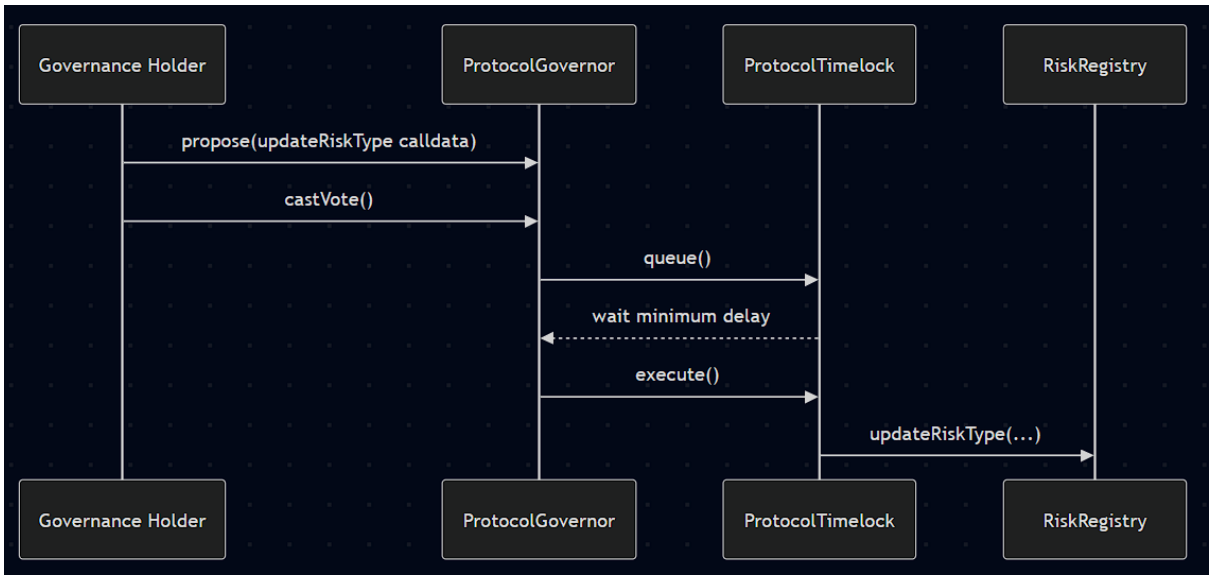



Security-relevant invariants:

- only the owner or an approved operator of the policy NFT can trigger a claim;
- the policy must still be active and unexpired;
- stale or incomplete oracle rounds revert in the adapter;

- state changes in the pool happen before the vault payout, preventing double-claim reuse of the same coverage reservation.

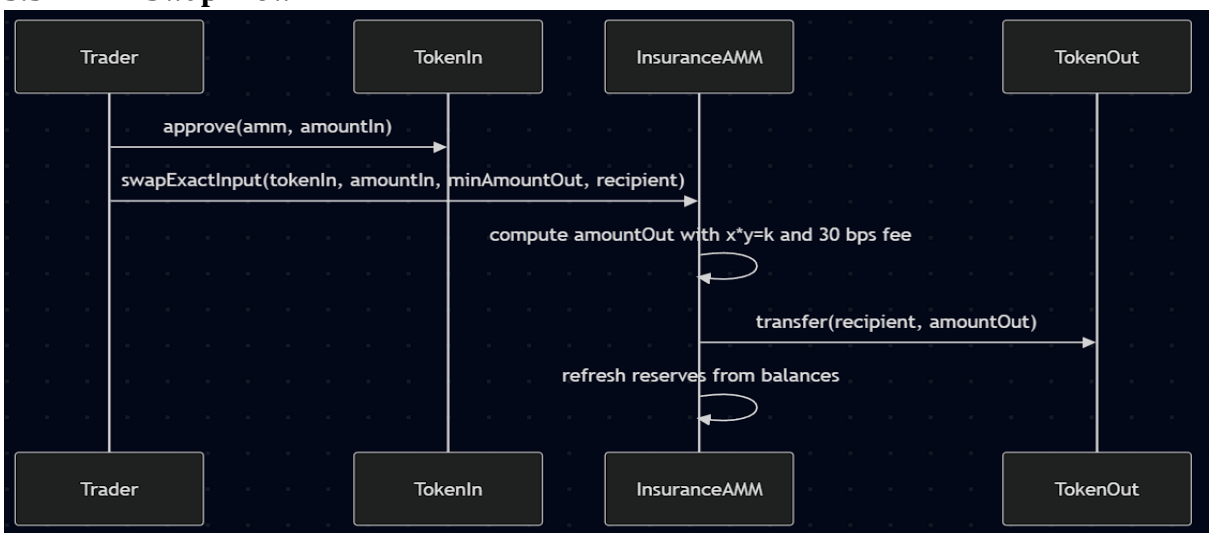
### 3.4 Governance Parameter Update Flow




Security-relevant invariants:

- governance weight comes from `ERC20Votes` checkpoints in `GovernanceToken`;
- proposals must satisfy `proposalThreshold = initialSupply / 100`;
- successful proposals still wait through a `2 days` timelock before execution;
- if bootstrap admin roles remain externally controlled, they can weaken this flow, which is discussed in the audit report.

### 3.5 AMM Swap Flow



Security-relevant invariants:

- only `token0` or `token1` may be used as `tokenIn`;
- `minAmountOut` enforces user-defined slippage control;
- reserves are tracked explicitly and refreshed after each state-changing action;
- the AMM is a separate module and does not directly collateralize claim risk.

## 4. Data Model and Storage Layout

### 4.1 Storage Reading Guide

- `constant` variables are compile-time values and do not occupy mutable storage.
- `immutable` variables are embedded in deployed bytecode and also do not occupy regular mutable storage slots.
- Mappings and dynamically sized data structures store content at hashed storage locations derived from a declared root slot.
- OpenZeppelin base contracts bring inherited storage. For non-upgradeable contracts this matters for reasoning, but storage-collision proofs are only mandatory for the proxy-based upgrade path.

### 4.2 Domain Structs

**`RiskRegistry.RiskType`**

| Field | Meaning |
|---|---|
| `name` | human-readable risk label |
| `premiumRateBps` | annualized premium rate in basis points |
| `maxCoverage` | per-risk cap for concurrently active coverage |
| `oracleFeed` | Chainlink feed address used for claim checks |
| `triggerThreshold` | raw oracle answer threshold used in comparisons |
| `stalenessLimit` | maximum acceptable feed age in seconds |
| `active` | whether new policies for the risk can be sold |

**`InsurancePool.Policy`**

| Field | Meaning |
| --- | --- |
| `policyId` | monotonically increasing primary key |
| `buyer` | owner at purchase time and payout recipient |
| `riskTypeId` | reference into `RiskRegistry` |
| `premium` | premium amount paid into the vault |
| `coverageAmount` | insured amount reserved against vault liquidity |
| `startTime` | purchase timestamp |
| `endTime` | expiration timestamp |
| `status` | `Active`, `Claimed`, or `Expired` |

## 4.3 Contract-by-Contract Storage Layout

| Contract | Inherited Mutable Storage | Project-Defined Persistent Storage | Immutable / Constant / Stateless Notes |
| --- | --- | --- | --- |
| `GovernanceToken` | `ERC20` balances and allowances, `ERC20Votes` checkpoints and vote delegation data, `Ownable` owner, `ERC20Permit` nonces | none | `clock()` uses timestamp mode, but no extra custom storage is added |
| `PolicyNFT` | `ERC721` ownership/approvals, `Ownable` owner | `insurancePool`, `_baseTokenUri` | mint authority is controlled by `insurancePool`; metadata base URI is mutable by owner |
| `InsuranceVault` | `ERC20` share balances, `ERC4626` asset accounting, `Ownable` owner, `Pausable` flag, `ReentrancyGuard` status | `insurancePool`, `claimManager` | free liquidity is computed from external pool state, not stored independently |
| `RiskRegistry` | `AccessControl` role-admin mapping and role-member mapping | `riskTypeCount`, `_riskTypes` mapping | `RISK_MANAGER_ROLE` is a constant hash |
| `ChainlinkOracleAdapter` | none | none | stateless adapter; validates oracle data without persisting it |
| `InsurancePool` | `Ownable` owner, `Pausable` flag, `ReentrancyGuard` status | `claimManager`, `nextPolicyId`, `activeCoverage`, `_policies`, `activeCoverageByRisk` | `collateralAsset`, `_vault`, `_policyNFT`, `_riskRegistry` are immutable references |
| `ClaimManager` | `Ownable` owner, `ReentrancyGuard` status | none | all protocol dependencies are immutable references |
| `InsuranceAMM` | `ERC20` LP balances and allowances, `ReentrancyGuard` status | `_reserve0`, `_reserve1` | `token0` and `token1` are immutable; fee constants are compile-time values |
| `ProtocolTimelock` | `TimelockController` roles, operation queue, predecessor/dependency tracking, delay state | none | `MIN_DELAY_SECONDS` is constant; deployment flow currently uses open executors |

| | | | |
|---|---|---|---|
| `ProtocolGovernor` | inherited Governor proposal state, voting settings, quorum fraction settings, timelock integration state | none | constructor wires token, timelock, quorum fraction, threshold, and durations |
| `InsuranceFactory` | `Ownable` owner | none | owner can deploy arbitrary bytecode via CREATE/CREATE2 |
| `UpgradeableInsurancePoolV1` | `Initializable`, `UUPSUpgradeable`, and `OwnableUpgradeable` internal bookkeeping stored in the proxy | `collateralAsset`, `totalCoveredAmount`, `totalPoliciesSold`, `__gap[50]` | upgrade authorization is `onlyOwner` |
| `UpgradeableInsurancePoolV2` | inherits the entire V1 layout unchanged | `maxPolicyDuration` appended in the derived version | no existing slot is repurposed |
| `AssemblyBenchmark` | none | none | stateless helper for benchmark comparison |
| `IInsurancePool` | not applicable | none | interface only |

### 4.4 Storage Layout Details by Contract

**`GovernanceToken`**

- No project-specific mutable storage is added on top of OpenZeppelin bases.
- The important inherited state is:

- ERC-20 balances/allowances;
- vote delegation and checkpoints from `ERC20Votes`;
- `owner` from `Ownable`;
- permit nonces from `ERC20Permit`.

- Because the token changes the governance clock mode to timestamps, all off-chain and verification tooling must query past timepoints rather than the current one.

**`PolicyNFT`**

Ordered project-defined mutable state:

1. `insurancePool`
2. `_baseTokenUri`

Implications:

- the owner can change which pool is allowed to mint policy NFTs;
- metadata mutability is explicit and should be treated as an operational governance power rather than an immutable NFT guarantee.

**`InsuranceVault`**

Ordered project-defined mutable state:

1. `insurancePool`
2. `claimManager`

The vault does not persist `lockedLiquidity` or `freeLiquidity`; both are derived from live state. That keeps the accounting surface smaller, but it also means the vault trusts the pool's current `activeCoverage()` value.

**`RiskRegistry`**
Ordered project-defined mutable state:

1. `riskTypeCount`
2. `_riskTypes`

The mapping payload contains every claim-critical business parameter: premium, cap, oracle feed, trigger threshold, staleness limit, and active flag. This makes registry governance highly sensitive: a misconfigured risk entry can change payouts without touching `InsurancePool` or `ClaimManager` code.

**`InsurancePool`**
Ordered project-defined mutable state:

1. `claimManager`
2. `nextPolicyId`
3. `activeCoverage`
4. `_policies`
5. `activeCoverageByRisk`

The actual dependency references to vault, NFT, registry, and collateral token are immutable and therefore not part of the mutable storage layout. This is a useful safety property: once the pool is deployed, those addresses cannot be silently changed by writing storage.

**`ClaimManager`**
`ClaimManager` deliberately keeps no mutable protocol accounting of its own. It reads the policy, risk, and vault state from other contracts and only persists what its inherited `Ownable` and `ReentrancyGuard` parents require. The design

goal is to keep the payout trigger path auditable and narrow.

**`InsuranceAMM`**
Ordered project-defined mutable state:

1. `_reserve0`
2. `_reserve1`

LP balances, allowances, and total supply live in inherited ERC-20 storage.
The AMM therefore has two layers of state:

- inherited share-accounting state;
- custom reserve state used for swap math.

**`ProtocolTimelock`**
No new project-defined mutable slots are introduced. The important inherited timelock state contains:

- role memberships for `DEFAULT_ADMIN_ROLE`, `PROPOSER_ROLE`, `EXECUTOR_ROLE`,
and `CANCELLER_ROLE`;
- operation hashes and timestamps for queued calls;
- min-delay state.

This inherited storage is security-critical even though it is not custom code, because it defines who can schedule and execute privileged protocol changes.

**`ProtocolGovernor`**
No custom mutable slots are added, but the inherited Governor modules keep:

- proposal metadata and state;
- vote receipts and vote counting mode;
- proposal threshold and voting period settings;
- quorum fraction configuration;
- timelock linkage state.

The absence of custom storage reduces local complexity, but governance security still depends heavily on these inherited modules.

**`InsuranceFactory`**

No project-defined mutable storage is added beyond the inherited `owner`. The security implication is nevertheless large: whoever controls `owner` controls the ability to deploy arbitrary bytecode under the factory's authority.

**`UpgradeableInsurancePoolV1` and `UpgradeableInsurancePoolV2`**

V1 custom mutable storage:

1. `collateralAsset`
2. `totalCoveredAmount`
3. `totalPoliciesSold`
4. `__gap[50]`

V2 custom mutable storage:

1. inherited full V1 layout
2. `maxPolicyDuration`

### 4.5 Upgradeable Storage-Collision Proof

The only upgradeable path in this repository is the isolated `UpgradeableInsurancePoolV1`/`UpgradeableInsurancePoolV2` example behind an `ERC1967Proxy`. Storage-collision safety is argued as follows:

1. The proxy stores its own admin/implementation metadata in the reserved EIP-1967 slots managed by OpenZeppelin. These slots are outside the implementation's ordinary storage layout by standard design.
2. V1 establishes a stable inheritance order: `Initializable -> UUPSUpgradeable -> OwnableUpgradeable -> project fields`. Changing that order in a future version would be unsafe, but V2 does not do so.
3. V1 defines three custom variables and then reserves `uint256[50] __gap`. No existing custom slot is reused or type-changed in V2.
4. V2 only appends `maxPolicyDuration` in the derived contract. It does not overwrite any earlier field, remove the gap, or reorder parents.
5. Therefore V2 is append-only from the perspective of persistent state. The reserved gap is left conservative rather than being resized, which is less space-efficient but still storage-safe.

**Conclusion:**
Under the concrete V1-to-V2 change shipped in this repository,
storage collisions are not introduced by the implementation code. The remaining operational risk is not slot collision but owner control over the proxy upgrade path if governance handoff is incomplete.

## 5. Trust Assumptions

### 5.1 Actor and Power Matrix

| Actor | What the actor can do | What goes wrong if the actor is malicious or compromised |
|---|---|---|
| Policy buyer / holder | buy policies, trigger claims for owned or approved NFTs | can only affect own funds and transaction ordering |
| Underwriter / LP | deposit and withdraw free liquidity | can withdraw only what reserve-aware vault rules allow |
| `ClaimManager` contract | call `markPolicyClaimed` and `payClaim` after checks | if rewired to a malicious address, payout flow can be redirected |
| `RISK_MANAGER_ROLE` holder | add/update/deactivate risk definitions | can enable bad feeds, bad thresholds, or impossible premiums |
| `ProtocolTimelock` | execute owner/role-gated protocol changes after delay | compromise can rewrite admin-controlled protocol behavior |
| `ProtocolGovernor` majority | queue timelocked changes | majority capture can steer governance-approved mutations |
| Bootstrap owner / admin EOA | initial wiring, role grants, ownership transfers, possibly proxy upgrades | if retained too long, governance can be bypassed or weakened |
| Chainlink feed operator/network | publish oracle data and timestamps | incorrect data can approve or reject claims incorrectly |
| Frontend / subgraph maintainers | influence usability and monitoring | cannot change final on-chain truth but can mislead users operationally |

### 5.2 Timelock Powers

Once ownership and risk roles are handed off correctly, the timelock can:

- call owner-only functions on `PolicyNFT`, `InsuranceVault`, `InsurancePool`, and `InsuranceFactory`;
- exercise `RISK_MANAGER_ROLE` on `RiskRegistry`;
- act as the intended governance execution boundary for privileged changes;
- if given proxy ownership operationally, authorize upgrades of the example UUPS module.

The timelock is therefore the most important control concentration point in the designed steady state.

### 5.3 Individual Admin Powers

Before full governance handoff, the bootstrap owner/admin can:

- wire `insurancePool` into the NFT and vault;
- wire `claimManager` into the vault and pool;
- pause/unpause vault and pool;
- grant or revoke risk-management permissions;
- deploy arbitrary code through the factory;

- retain proxy ownership for the upgradeable example;
- if timelock admin is not renounced, grant privileged timelock roles.

This is acceptable during setup only if it is treated as temporary and audited through a strict release checklist.

### 5.4 Multisig Compromise Scenario

The current repository does not hardcode a multisig address on-chain. However, if the team operationally places a multisig in any of these bootstrap roles, the multisig simply inherits the same powers as the current EOA owner/admin.

If such a multisig is compromised:

- it can pause protocol modules that still trust `owner`;
- it can repoint `claimManager` or `insurancePool` in modules where wiring is mutable;
- it can grant timelock roles if it still holds `DEFAULT_ADMIN_ROLE`;
- it can upgrade the UUPS example if it still owns the proxy;
- it can deploy malicious bytecode through `InsuranceFactory`.

Therefore the security question is not "EOA vs multisig" but "what privileges exist at all after bootstrap, and have they been renounced or handed to the timelock?".

### 5.5 External Dependency Assumptions

- Chainlink feeds must be live, honest, and semantically appropriate for each insured risk.
- Base Sepolia finality and RPC availability are operational assumptions for demos and monitoring.
- The subgraph, if deployed later, must correctly index emitted events; it is not a source of protocol truth.
- The frontend build passing is not the same thing as full wallet-based production validation.

## 6. Design Decisions Log (ADR Set)

### ADR-01: Use an ERC-4626 vault for collateral accounting

Context: The protocol needs share accounting for underwriters, a standard deposit/withdraw interface, and a clear distinction between free and reserved liquidity.

Options considered:

- a bespoke share-accounting contract;
- an ERC-20 plus custom deposit math;
- ERC-4626 over the collateral token.

Decision: Use ERC-4626 and add reserve-aware withdrawal restrictions.

Consequences: The accounting surface is smaller and more standard, but reserve logic still needs explicit custom checks through `freeLiquidity()` and `lockedLiquidity()`.

**ADR-02: Separate claim execution from policy purchase**

Context: Policy purchase logic and oracle-triggered payout logic evolve at different rates and carry different audit risks.

Options considered:

- place all logic in `InsurancePool`;
- put claim execution in a helper contract;
- rely on off-chain payout decisions.

Decision: Keep claim evaluation and payout triggering in `ClaimManager`.

Consequences: The claim path becomes easier to reason about, but wiring the `claimManager` address becomes a privileged setup step that must be controlled.

**ADR-03: Put risk parameters in a governance-managed registry**

Context: New insurable assets, premium curves, and oracle feeds should not require redeploying the whole core protocol.

Options considered:

- hardcode one feed and one policy type;
- deploy a new pool per asset;
- centralize risk definitions in `RiskRegistry`.

Decision: Use `RiskRegistry` with `RISK_MANAGER_ROLE`.

Consequences: Governance becomes more flexible, but misconfiguration risk moves into the registry and must be managed by role controls and audit procedures.

**ADR-04: Use Governor plus Timelock instead of permanent owner-only control**

Context: The course project needs to demonstrate DAO-style control rather than a single admin wallet with permanent authority.

Options considered:

- keep owner-only authority forever;
- use a plain multisig only;
- use `ProtocolGovernor` plus `ProtocolTimelock`.

Decision: Build the intended final admin path around Governor plus Timelock.

Consequences: Governance is transparent and delayed, but bootstrap operations must still explicitly renounce any temporary admin powers.

**ADR-05: Keep the upgradeability example separate from the main pool**

Context: The assignment requires upgradeability understanding, but a course-project insurance core is already complex without mixing proxy semantics into every business rule.

Options considered:

- make the main pool upgradeable from day one;
- omit upgradeability entirely;
- create an isolated UUPS example module.

Decision: Keep `UpgradeableInsurancePoolV1/V2` as a separate example.

Consequences: Storage-collision reasoning becomes tractable and demonstrable, but the example still needs a proper governance handoff if it is included in a real deployment story.

**ADR-06: Use timestamp-based governance checkpoints**

Context: Governance settings are expressed in human time (`1 day`, `1 week`), while default vote checkpoints are often block-number based.

Options considered:

- block-number checkpoints;
- timestamp checkpoints via overridden `clock()`.

Decision: Override `GovernanceToken.clock()` to use timestamps.

Consequences: Voting windows match wall-clock semantics more naturally, but tooling and verification code must query past timepoints, not the current one.

**ADR-07: Treat frontend and subgraph as integration layers, not trust anchors**

Context: The project includes a React frontend and subgraph assets, but core correctness should still be on-chain.

Options considered:

- move business logic off-chain;
- make frontend/subgraph convenience layers only.

Decision: Keep contract logic authoritative and use frontend/subgraph for access and indexing.

Consequences: The system remains verifiable from chain state, but end-user UX still depends on the quality of off-chain tooling.

**7. Residual Architecture Limitations**

- The production frontend build passes, but full wallet-based live interaction still needs a funded testnet wallet and manual end-to-end verification.
- The subgraph manifest is wired to the recorded Base Sepolia deployment, but a hosted subgraph endpoint is not claimed unless deployed separately.
- The core protocol ownership handoff to the timelock is represented in the verified deployment workflow, but operational governance hygiene still depends on revoking any bootstrap roles that should not persist.
- The AMM and insurance pool are adjacent modules rather than one tightly integrated solvency engine. This is acceptable for the course project, but it should be described honestly during demonstration.